CSC209 Assignment 3 Mini-Project

Project Title: Parallel Word Count (`pwc`)
Category: 1 - Multi-process application using pipes
Members: Jimmy Wu, Elaine Xing, Geo Lee

5.1 Project Overview

Our program, `pwc`, is a Category 1 multi-process program designed to count word frequencies in multiple text files concurrently. Instead of processing each text file one by one, the program utilizes a child process per file to count the word frequencies in parallel.

The program takes a text file containing a list of filenames as input. The parent process reads the list and forks a worker process for each file (`main.c`, lines 280-285). A worker process counts word frequencies in its assigned file and generates a linked list of words with frequency using `generate_node_family` (`main.c`, lines 351-360), which is defined in `helper.c`. This list is then converted into a continuous string in `convert_node_family` (`main.c`, lines 362) in `helper.c` and sent to the parent process via a pipe using `send_result_message` (`main.c`, line 374). The parent reads each child's result from its pipe by calling `read_result_message` (`main.c`, line 407) and immediately merges the reported word counts into a global linked list using `merge_child_results` (`main.c`, line 426). To produce the final output, the parent copies pointers to the merged linked-list nodes into an array, sorts the array with `qsort`, and then prints the results (`main.c`, lines 448-480).

The user interacts with the program using the command line. The user provides the text file containing a list of filenames as an argument. As well, the user can customize the output by providing flags anywhere in the command: (`-f`, `-rf`, `-a`, -ra) for sorting order, (`-i`) for case-insensitivity, (`-m N`) for a minimum frequency filter, and (`-k K`) for a top-results filter. If no flags are found in the command, the words are printed in high-to-low frequency order by default.

5.2 Build Instructions

The project includes a `Makefile`.

To compile:

Run `make` in the project directory using the provided Makefile.

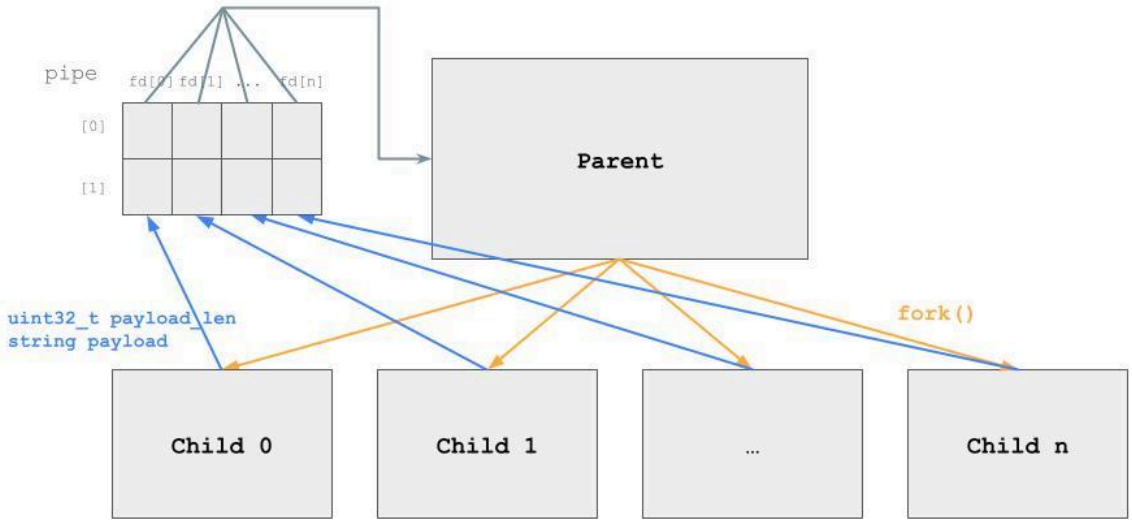
To run:

```
./pwc [-f | -rf | -a | -ra] [-i] [-m N] [-k K] <filename.txt>
```

Command-line arguments:

- `<filename.txt>`: (REQUIRED) A text file containing the list of paths to the files to be processed. The file list must contain at least 3 input filenames, one per line.
- `-f`: Sorts the output by high-to-low frequency. This option is selected as default in case no flags are stated.
- `-rf`: Sorts the output by low-to-high frequency.
- `-a`: Sorts the output in alphabetical order (A-Z)
- `-ra`: Sorts the output in reversed alphabetical order (Z-A)
- `-i`: Ignore the case when counting words and any word is counted towards lowercase. (APPLE, APple, ApPlE, apple are considered the same)
- `-m N`: Minimum count filter. Only prints words with frequency at least N
- `-k K`: Top filter. Only prints the top K words after sorting

5.3 Architecture Diagram



5.4 Communication Protocol

Field	Description
Direction	Worker → Parent
Encoding	A length-prefixed message. In the function `send_result_message` (`main.c`, lines 135-139), the worker first writes a 4-byte **`uint32_t`** `payload_len`, then writes exactly `payload_len` bytes of `payload`. For normal results, the payload is a newline-separated string where each line is formatted as `"%s %d\n"`. For worker-side errors, the payload begins with the `ERROR_PREFIX` macro (`main.c`, line 13) followed by a human-readable error message in the function `send_child_error_and_exit` (`main.c`, lines 141-151).
Semantics	In the function `read_result_message` (`main.c`, lines 153-172), the parent process first reads 4 bytes to find the `payload` size, then dynamically allocates a buffer of `payload_len + 1` bytes. The parent loops its `read()` calls until it has read the whole payload. If the payload begins with the `ERROR_PREFIX` prefix, the parent prints the remainder of the string to `stderr`, waits for the children, cleans up allocated memory, and exits. Otherwise, it adds a null-terminator at the end of the buffer. Then, the parent passes the buffer to `merge_child_results` (`main.c`, lines 59-94), which splits the copied version of the buffer using `strtok(copy, "\n")` and extracts the word and frequency using `sscanf(line, "%79s %d", word, &count)`.
Error handling	`write()` and `read()` calls are wrapped in custom helper functions, `write_all()` (`main.c`, lines 96-112) and `read_all()` (`main.c`, lines 114-133). These functions loop continuously to handle partial reads and writes and safely retry the task if interrupted by a signal (checking for `errno == EINTR`).

5.5 Concurrency Model

The program achieves concurrency by allowing all child processes to process their assigned files concurrently.

Forking:

The parent process determines the number of target files and stores this value in an integer variable named `len` (`main.c`, lines 280-285). It creates pipes and forks according to `len`, using a loop (`main.c`, lines 326-338). Since the parent does not call `wait()` in the loop, all workers are immediately spawned and start processing assigned files concurrently.

Reading:

The parent does not use a separate "ready" signal. A worker is effectively ready when it writes its length-prefixed result message to its pipe. Once all workers are spawned, the parent iterates through the pipes to read their outputs sequentially (`main.c`, lines 406-428). Because the OS buffers pipe data automatically, children can write their results concurrently without data loss, even if the parent is currently reading from a different child's pipe. The parent loops until it has read the complete payload from each child's pipe.

Reaping:

Finally, the parent process enters a third loop (`main.c`, lines 431-445) using `waitpid()` to collect all children's exit statuses, ensuring that no zombie processes are left behind.

5.6 Error Handling and Robustness

1. **Invalid File Handling:** If the parent cannot open the file list, it reports the error with `perror()` and exits (`main.c`, lines 273-278). If a child cannot open its assigned input file, it sends an ERROR_PREFIX message through its pipe, and the parent prints the error, cleans up memory, waits for remaining children, and exits (`main.c`, lines 352-354)
2. **System Call Failures:** The return values of every crucial system call are explicitly checked. If there is an error, the program cleans up allocated memory and exits. For instance, if `pipe()` (`main.c`, lines 332-336) and `fork()` (`main.c`, lines 339-347) fail when spawning workers, it stops and does the cleaning rather than attempting to proceed.
3. **Variable-length messages:** Because worker results can vary in size, each child sends the payload length first in `send_result_message` (`main.c`, lines 135-139). This allows the parent to allocate a correctly sized buffer and avoids truncating or over-reading message data in `read_result_message` (`main.c`, lines 153-172).
4. **Resource Leaks:** We strictly manage file descriptors: the children closes its unused pipe ends right after `fork()` in `close_child_pipe_ends` (`main.c`, lines 348). As well, we ran the program through `valgrind` and ensured that all dynamically allocated memory is freed normally.
5. **Interrupted System Calls:** The helper functions `write_all()` (`main.c`, lines 96-112) and `read_all()` (`main.c`, lines 114-133) retry `write()` and `read()` when interrupted by a signal (`errno == EINTR`). In addition, `wait_for_children()` (`main.c`, lines 201-212) retries `waitpid()` when interrupted. Therefore, when these calls are interrupted, the program safely makes another attempt for the system call.
6. **Child-to-Parent Error Propagation:** If a child process encounters an error, it sends a message prefixed with `ERROR_PREFIX` through the pipe, rather than writing to `stderr`, using `send_child_error_and_exit` (`main.c`, lines 353 and 359). This allows the parent to clean up all the allocated memory and remaining children so that `exit()` can be done safely.
7. **Invalid command-line arguments:** The program detects a missing file-list argument, missing values after `-m` or `-k`, and invalid numeric values for these flags. In these cases, it prints an error message and exits rather than continuing with invalid parameters. (`main.c`, lines 249-262)
8. **Incomplete pipe message:** If a child terminates or closes its pipe before sending the full length-prefixed payload, the parent detects this when `read_result_message()` (`main.c`, line 155-157) fails to read the expected number of bytes. It then prints an error, waits for the remaining children, frees allocated memory, and exits safely.

5.7 Team Contributions

Jimmy:
In `main.c`, I implemented the parent-process side of the program, including reading worker results from pipes, merging child output into the global word-count list, and handling child cleanup and abnormal exits. I also implemented and improved the pipe communication protocol through functions such as `write_all()`, `read_all()`, `send_result_message()`, and `read_result_message()`, and added worker-to-parent error reporting using `ERROR_PREFIX`. I also added support for case-insensitive counting and the `-m` and `-k` filters, along with several cleanup and robustness fixes for file descriptor handling and error paths.

Elaine:
In `main.c`, I implemented the code to read the input file from argv and count how many individual file names are in this input file. This determines how many child processes will be forked later. I also wrote the code for each child process that will open and read their associated file. The helper functions in `helper.c` will read each word from the opened file and parse it into an array of strings. From this array, other helper functions create a linked list of structs that store the word and the number of times that word has been encountered in the file. I then convert the linked list into a text format of word-frequency\n, write that into the pipe and allow the parent process to read the word frequencies from each child process.

Geo:
In `main.c`, I implemented the custom sorting feature and command-line argument parsing, which allows users to specify the output order (`-f`, `-rf`, `-a`, `-ra`). To make this feature work efficiently, I added custom comparison functions `compare_...()` and utilized `qsort()` to sort an array of node pointers prior to final output. Also, I was responsible for the documentation of this project, including the system architecture diagram, and for the demo video.